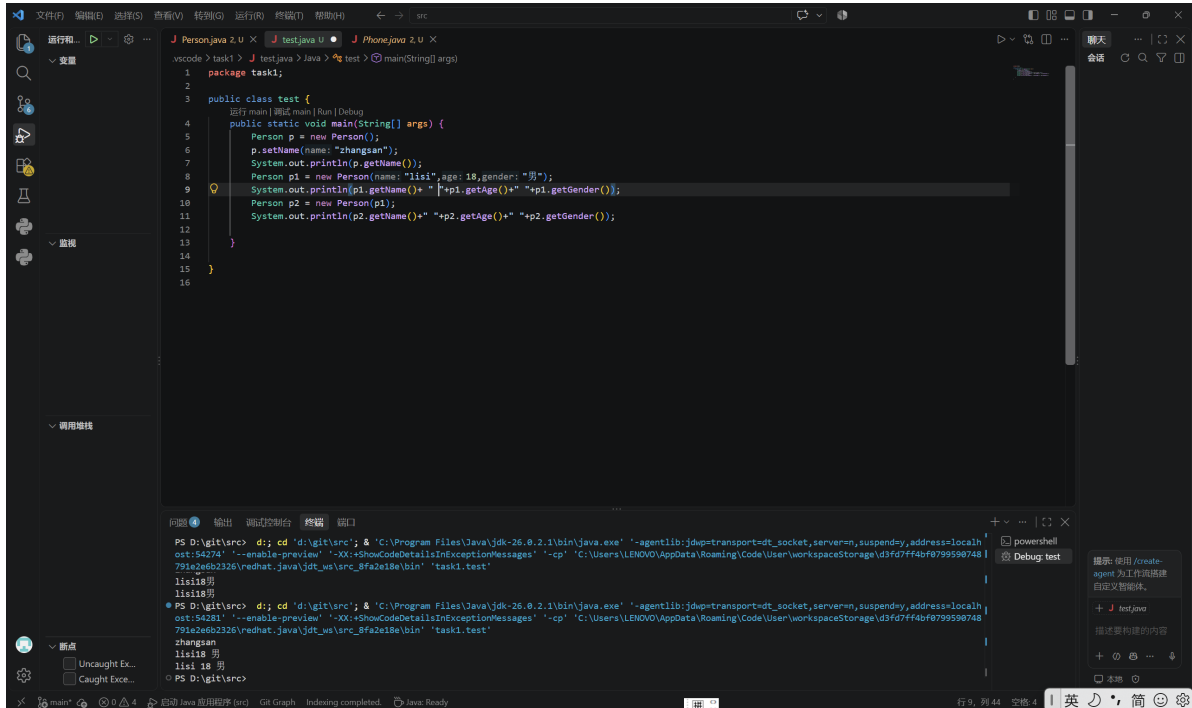


3：面向对象、封装继承与多态

Task1

1.



```
1 package task1;
2
3 public class test {
4     @SuppressWarnings("unused")
5     public static void main(String[] args) {
6         Person p = new Person();
7         p.setName("zhongsan");
8         System.out.println(p.getName());
9         Person p1 = new Person(name: "lisi", age: 18, gender: "男");
10        System.out.println(p1.getName() + " " + p1.getAge() + " " + p1.getGender());
11        Person p2 = new Person(p1);
12        System.out.println(p2.getName() + " " + p2.getAge() + " " + p2.getGender());
13    }
14 }
15
16 }
```

Output:

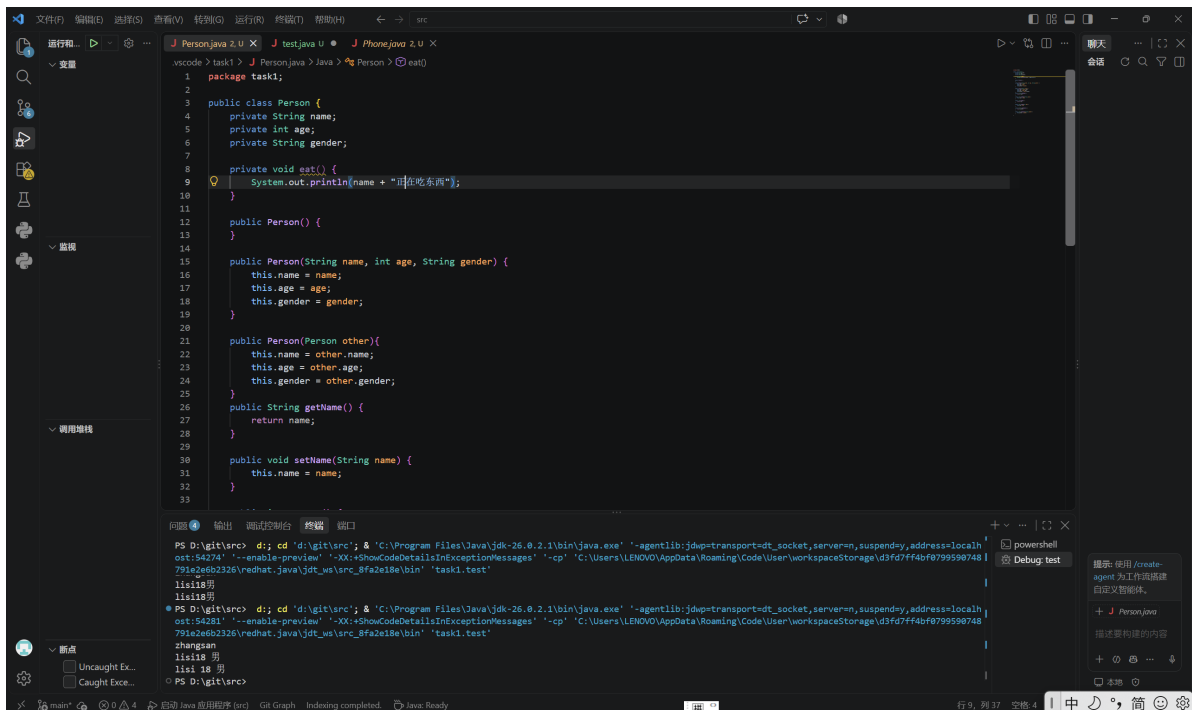
```
lisi18男
lisi18 男
zhongsan
lisi18 男
lisi 18 男
```

2.用到了this（当然还有other）

this实现了将局部变量的值赋给成员变量

（other实现了对象的复制）

3.



```
1 package task1;
2
3 public class Person {
4     private String name;
5     private int age;
6     private String gender;
7
8     private void eat() {
9         System.out.println(name + "正在吃东西");
10    }
11
12    public Person() {
13    }
14
15    public Person(String name, int age, String gender) {
16        this.name = name;
17        this.age = age;
18        this.gender = gender;
19    }
20
21    public Person(Person other) {
22        this.name = other.name;
23        this.age = other.age;
24        this.gender = other.gender;
25    }
26
27    public String getName() {
28        return name;
29    }
30
31    public void setName(String name) {
32        this.name = name;
33    }
34 }
```

Output:

```
lisi18男
lisi18 男
zhongsan
lisi18 男
lisi 18 男
```

4.

对象：解决问题需要的工具

类：对象的设计图纸，且必须先设计类才能获取对象，即有了图纸才能创造工具

Task2

1.一个尝试访问代码同Task1

2.public：所有类都可以访问

private：仅当前类的内部可以访问

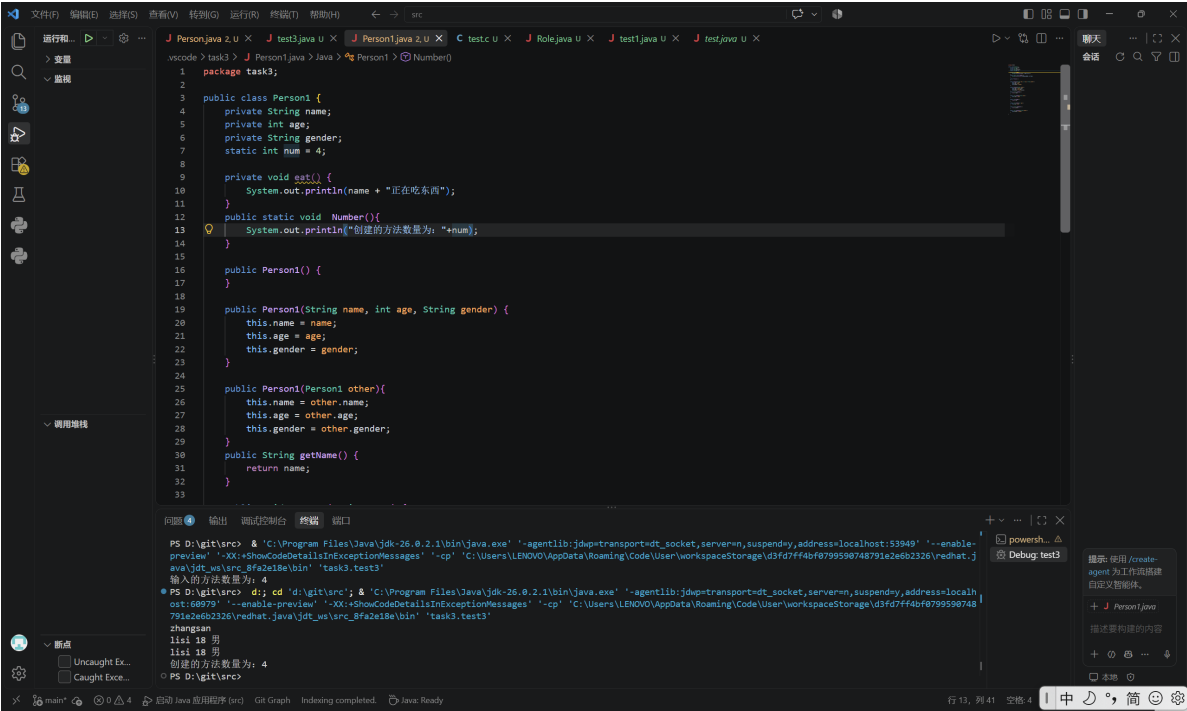
protect：仅不同包下的无关类无法访问

如果空着不写（或者说是默认状态），则只能在同一个类中访问

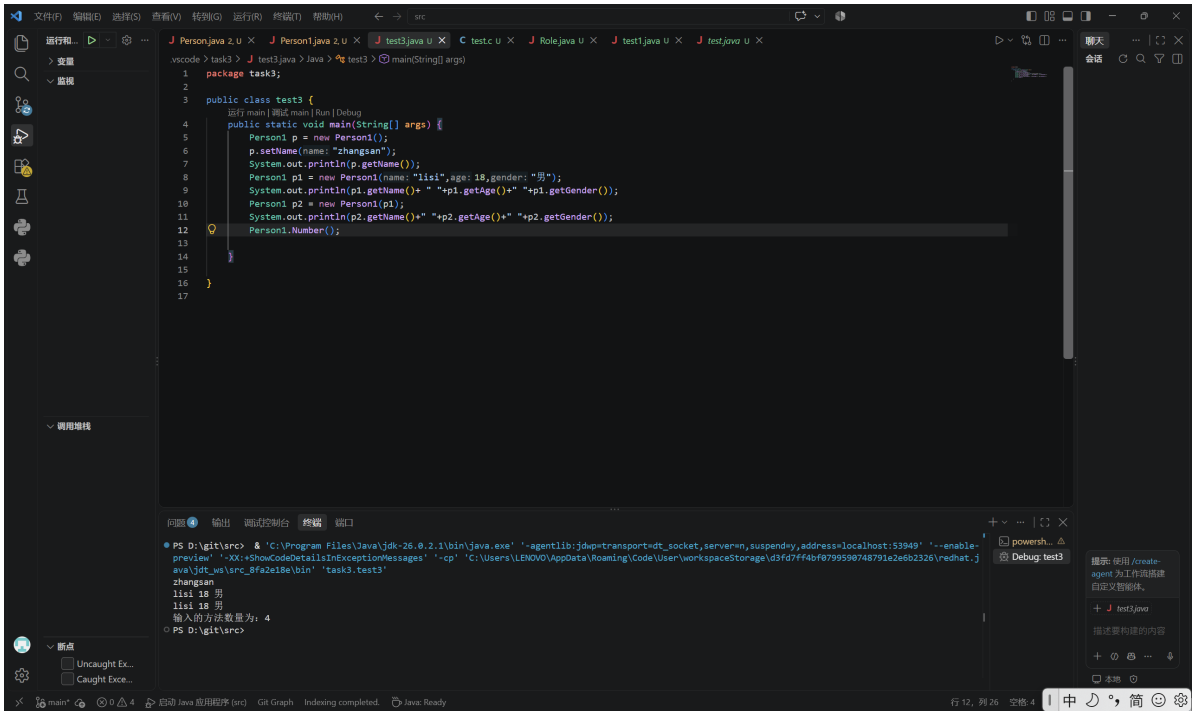
3.各种修饰符的访问权限如下表所示：

修饰符	同一个类中	同一个包其他类	不同包下的子类	不同包下的无关类
private	√			
空着不写	√	√		
protected	√	√	√	
public	√	√	√	√

4.



![image-20260924162302329](C:\Users\LENOVO\AppData\Roaming\Typora\typora-user-images\image-20260924162302329.png)



5. (1)

以这里的Person1类为例，JVM虚拟机在加载这个类的时候，将静态方法的字节码存放在元空间，它没有this参数，不属于任何对象，属于整个类的信息，所以可直接由类名调用。

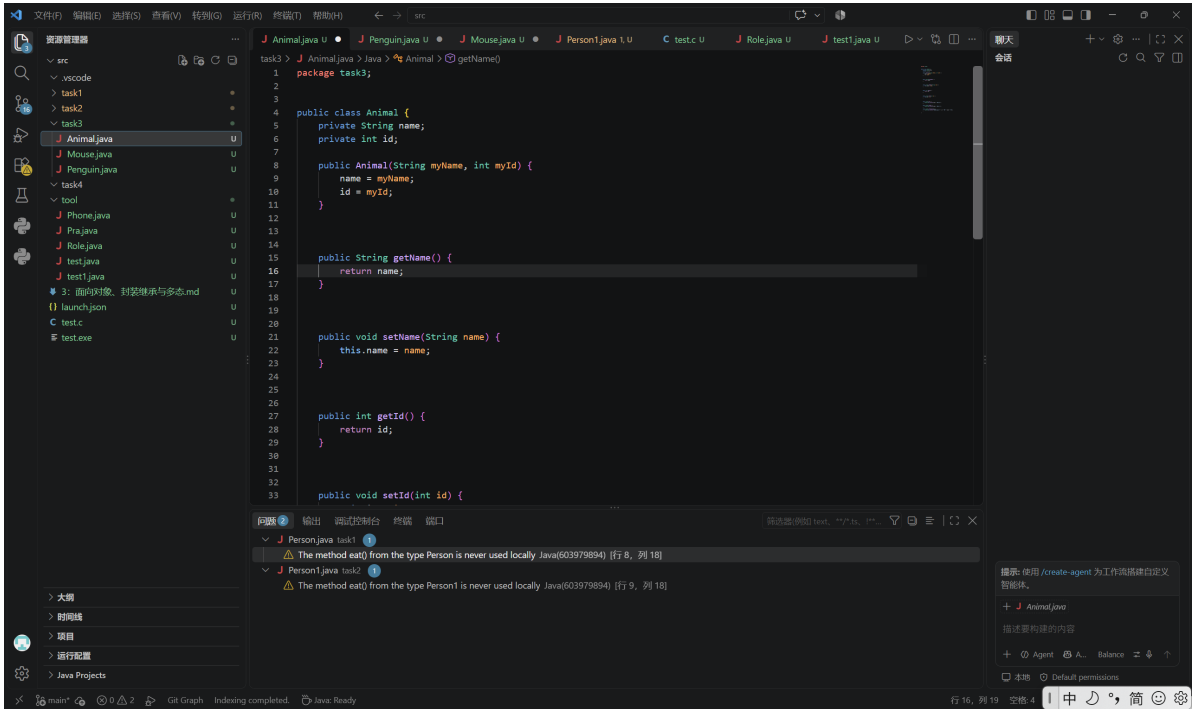
(2)

实例方法有一个隐藏this参数，就必须传一个对象进去，所以只能先new一个对象出来，才能调用该方法。否则this就没有值，实例方法根本不知道访问哪个对象的变量。

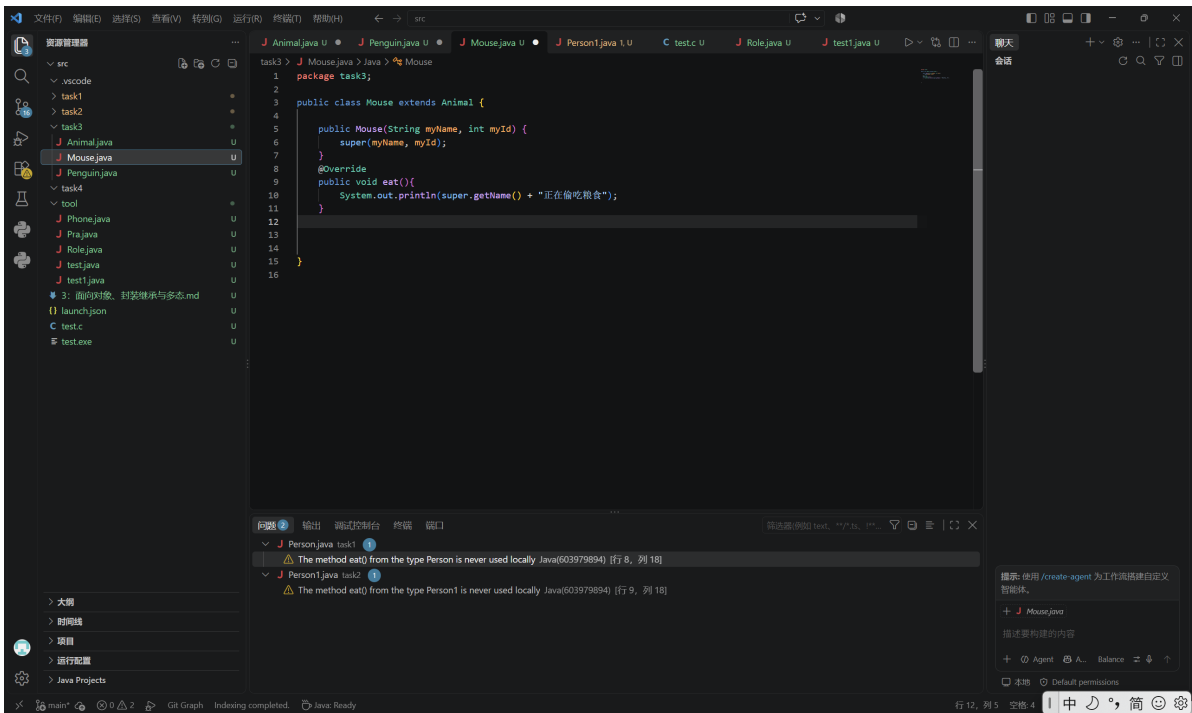
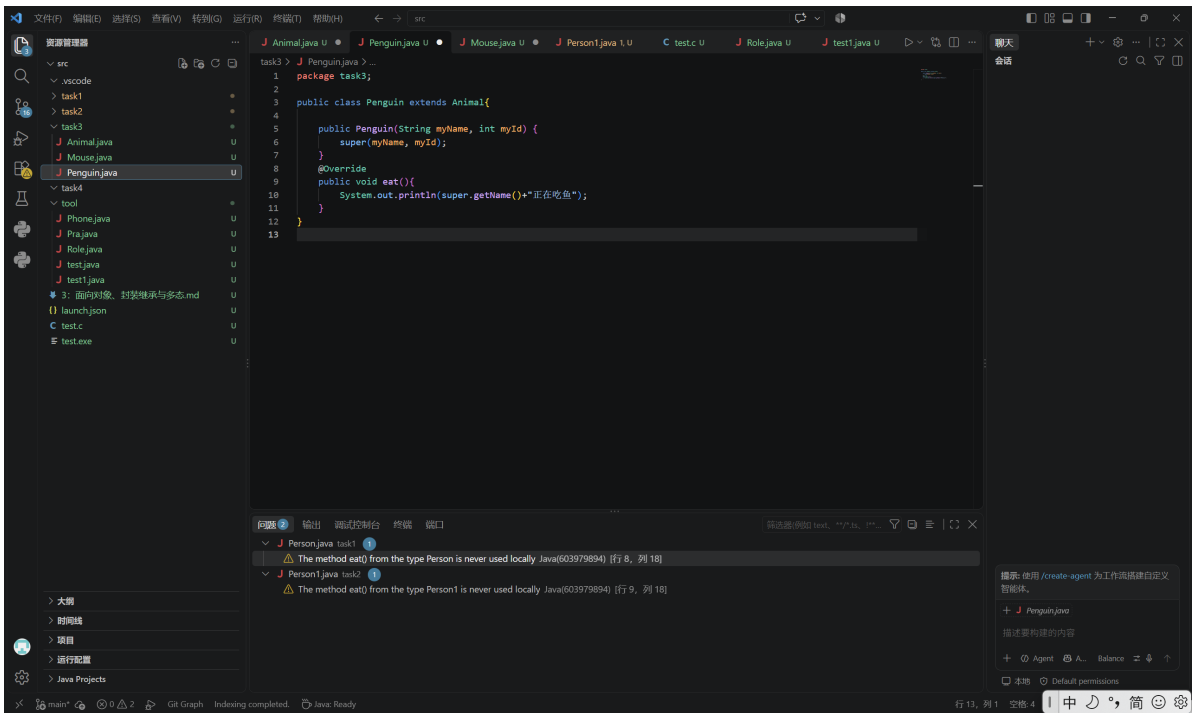
Task3

1.

父类如图所示（显示不完，可在文件中查看源代码）：



2.+3.两个子类分别如图所示：



4.

java中，如果一个子类同时继承了两个父类，若此时这两个父类中有同名的方法，那么在子类中创建对象调用这两个方法时就会发生冲突，即菱形冲突（简单来说，B和C都继承自A，如果B、C重写了A的同一个默认方法，D实现B、C时就产生菱形冲突，源自c++的概念）

Task4

抽象方案在task4a，接口方案在task4b。

使用场景：接口适用于子类中的方法不完全相同时，需要接口来定义规则，实现子类方法的统一。

抽象则适用于子类方法格式统一但不确定方法体、必须强制重写时。

Task5

public：所有类都可以访问，即公开信息

protected: 仅相关类可以访问, 给信息访问增加了权限

private: 仅本类可以访问, 使信息无法被直接访问

修改后的代码大致如下 (基本符合常规适用场景)

```
public class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private String password;

    public BankAccount(String accountNumber, String accountHolder, double
initialBalance, String password) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        if (validateAmount(initialBalance)) {
            this.balance = initialBalance;
        } else {
            this.balance = 0;
        }
        this.password = password;
    }

    public void deposit(double amount) {
        if (validateAmount(amount)) {
            balance += amount;
        }
    }

    public boolean withdraw(double amount, String inputPassword) {
        if (!validatePassword(inputPassword)) {
            return false;
        }
        if (!validateAmount(amount)) {
            return false;
        }
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public boolean transfer(BankAccount recipient, double amount, String
inputPassword) {
        boolean success = this.withdraw(amount, inputPassword);
        if (success) {
            recipient.deposit(amount);
        }
        return success;
    }

    public double getBalance() {
        return balance;
    }
}
```

```
    public String getAccountInfo() {  
        return "账号: " + accountNumber + ", 户名: " + accountHolder + ", 当前余额: "  
+ balance;  
    }  
  
    private boolean validatePassword(String inputPassword) {  
        if(inputPassword == null){  
            return false;  
        }  
        return this.password.equals(inputPassword);  
    }  
  
    private boolean validateAmount(double amount) {  
        return amount > 0;  
    }  
}
```